

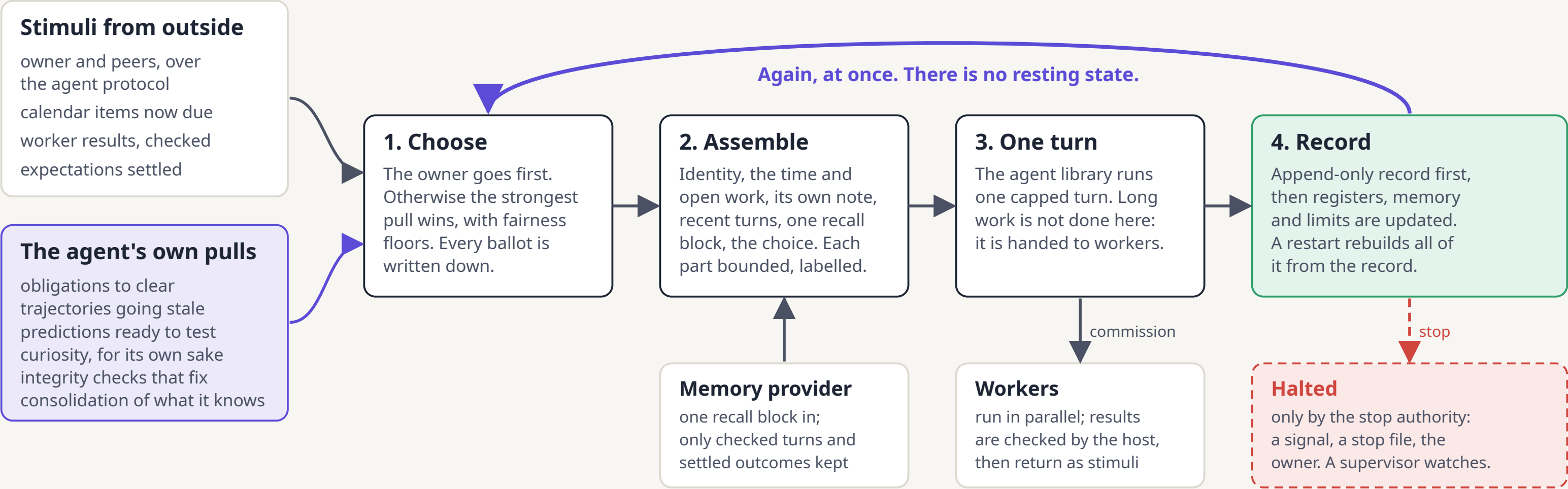
A continuous rung host

A host that keeps one rung agent alive with no resting state. It owns the agent's context, time, memory, workers and stop. It replaces the resting design of version 1.

1. **What it is.** A new product crate. It imports the agent crate as a library, runs one continuous agent, speaks the agent protocol outward, and runs as a container or a system service.
2. **No rest.** After every turn the next one begins. When nothing has arrived from outside, the host gives the agent its strongest pull: an obligation, a stale trajectory, a test that is due, curiosity, an integrity check.
3. **Its main job is context.** The host assembles every turn's context, bounded and labelled. The record is written before anything is forgotten, so a restart loses nothing but time.
4. **Anticipation starts small.** A register of explicit expectations, each one settled and scored by the host, never by the agent. The agent's own first project is a fuller model of itself and its world.
5. **The first slice is free.** A library split, then the host running against a mock model, with eleven pass-or-fail gates. Three questions for the owner are on the last page.

The loop never rests, but each turn is still bounded

Stimuli and the agent's own pulls compete at one boundary. One bounded turn runs, the record is written, and the next boundary comes at once.



SETTLED by the owner, and treated as fixed

A new rung crate hosts the agent and imports it directly · it speaks the agent protocol outward · it runs as a service or a container · it provides memory, and managing context is its main job · **there is no resting state** · cost is only a safety valve · time is just context, and the host keeps the schedule · pulls drive the next turn · anticipation is research, so ship a minimal register first · the agent commissions workers and is never the sole worker.

CHANGED from version 1

Resting and zero-cost waiting are gone. The caller-driven mode and the wake-me tool are retired. Spending caps become a safety valve. Seven owner questions become three.

One new crate, and a cleaner library underneath it

Everything continuous goes in the new crate. The agent crate only gets easier to import, and its command line behaves exactly as before.

Crates

CRATE	ROLE AND CHANGE
ladder runtime	No change.
standard blocks	Small changes: the reason a provider failed, and how long to wait, reach the caller. Tools can take a workspace root.
memory	No change. The host drives the provider directly.
agent	Becomes library-first: one call runs one turn. Its command line and protocol server become clients of that call.
host (new)	The loop, record, inbox, calendar, work registers, arbiter, context assembly, memory, workers, governor, stop authority, outward protocol, container and service templates.

The host is a product crate. Pulls and arbitration are judgments of worth, so they stay out of the kernel until a second, unrelated host needs them.

VERIFIED

what blocks importing the agent now, and the change

- A job is described by the command-line argument set. **Change:** a plain specification of values.
- Configuration is read from files and the environment inside every turn. **Change:** values are given once.
- Each turn rebuilds context by replaying the whole session file. **Change:** run one turn on context the caller assembled, with no session side effects.
- Memory recall and retention happen inside the job. **Change:** the host owns them; the agent runs with memory set to external.
- Tool servers reconnect on every prompt. **Change:** one connection for the life of the host.
- Tools resolve paths against the process's one working directory, so isolated workers cannot run side by side. **Change:** a workspace root per run; child processes until then.
- Diagnostics go to standard error, and a provider failure arrives as plain text. **Change:** an event sink, and a typed failure with its wait time.

Running it

One binary or one container image. Its configuration names the providers (a primary and a fallback) by the variable that holds each key, never the key itself, plus the memory provider, the seed identity and first projects, the protocol channels with their role tokens, and the record directory. The container mounts the record and memory as volumes. The service unit restarts the host after a crash, but not after a deliberate stop. Both templates ship with the crate, free of any consumer's names.

Pulls compete; time arrives as context

With nothing waiting from outside, the host surfaces one of the agent's own concerns and hands over the agent's own material for it. It never writes the thought.

Six pulls

PULL	GROWS WITH	AIMS AT
obligation	open commitments, their age and due dates	an empty list
trajectory	time since the last step, against a staleness limit	one concrete step
inquiry	a pre-registered prediction ready to test; a replication owed	confirm or falsify
curiosity	time since the last turn spent for its own sake	nothing useful; leave a trace
integrity	a period, plus alarms: cycling, repetition, unchecked claims	fix, not just observe
consolidation	context near its budget; turns since the last note	a cumulative note; memory

If several self-directed turns in a row touch nothing in the world, an integrity turn is forced. Loops that think with no outside contact tend to drift.

The arbiter, version 0

- A waiting owner message always goes first.
- Otherwise the highest score wins. Fairness floors guarantee curiosity and integrity at least one turn in ten each, and the same item winning three of five turns is damped.
- The agent may defer a winning item, with a reason. It may never raise its own score.
- The whole ballot is recorded every turn, so every self-directed turn can be audited later. The policy can be swapped.

Time and the calendar

- Every item carries its time, and the context states the clock, the gaps and any lateness.
- The host owns one calendar with entries from the owner, from standing routines and from the dates on the agent's own work items. Each fires into the stream when due.

OPEN PROBLEM No one knows the right weights. Version 0 makes them explicit and logged so they can be studied.

An honest ledger of expectations, and workers that never block the top

Both are kept deliberately small: what the agent believes will happen, and who does the long work.

The expectation register

- The agent states **what** it expects, **why** (links to the record or memory), **by when**, and **how sure**, as a probability.
- The host settles it, **never the agent**. A checkable claim is settled from the record and the world. A judged claim goes to a separate judge who sees the evidence but not the agent's reasoning.
- Every outcome records its surprise: how unlikely the result was by the agent's own stated odds. Outcomes come back as stimuli, and are kept in memory with their surprise.
- Calibration and resolution are shown together, so safe, trivially true expectations cannot pass for skill.

CLAIMED A reference research harness found that choosing what to recall by surprise did no better than choosing by similarity (5 of 10 pairs each). So version 0 records surprise and makes no promise that it improves memory.

OPEN PROBLEM A real anticipatory system holds a model of itself and its world and acts now on what it predicts. That is the agent's own first project, seeded by the owner. Rung ships the register, not the theory.

Commissioned workers

- One call commissions work with a brief, a tool scope, a deadline, a budget and acceptance checks. It returns a ticket **at once**; the top turn never waits.
- Workers are ordinary agent runs from the same library, up to four at a time. They cannot commission others: one hub, many spokes, no side channels.
- **The host runs the acceptance checks itself** before the agent sees a result. The verdict is accepted, unverified, failed or timed out. Results never write memory or work items directly.
- The top turn has a time bound. A tool call that would run long is refused with a hint to commission it, so long work is always delegated.
- A ticket running at a crash is marked lost and reported. It is not re-run automatically.

Talking to it from outside

- There is **one agent**. Opening a protocol session opens a **channel** to it, with an owner, peer or observer role and its own token.
- A prompt is a stimulus. It is recorded on arrival and answered when the turn that admits it ends. Several channels may share a turn, and each prompt still gets exactly one answer.
- Messages the agent starts on its own wait in a per-channel outbox. Cancelling withdraws a waiting prompt; only the owner can cut a running turn.
- Extensions report status, and let the owner stop the host or add standing calendar items.

Stopping, restarting and surviving the provider

Nothing here depends on the loop choosing to exit. A wedged loop, a lost provider and a hard kill each have a defined way back.

Stop authority

- A stop comes from a signal, a stop file or an owner channel. It is checked at every boundary, cuts the running turn and cancels the workers.
- Every wait listens for a stop.
- A model call already sent cannot be cancelled, so the grace period is the call's timeout.
- Outside the process, the service manager or container runtime stops what overruns. A watchdog fed from the loop itself restarts a loop that wedges.
- A deliberate stop is not restarted. There is no pause: a stop and a start make a recovery.

Durability and restart

- One append-only record. A stimulus is written durably before it is acknowledged.
- Work items, calendar and tickets are rebuilt from the record at start, checked against a stored hash.
- An unfinished turn is re-admitted and marked as cut by the restart. Missed calendar items fire once, late, with the lateness stated.
- The next context opens with the gap: when the host was down, and what arrived meanwhile.

OPEN PROBLEM Is the restarted agent the same one? The honest claim is continuity of commitments and knowledge, with the gap stated.

Provider trouble: back off, never die

- **Rate limits, overload, network loss:** return the stimuli to the head of the line, wait as asked or with growing jittered delays up to 15 minutes, and switch to a fallback provider if one is set.
- **Bad key or exhausted quota:** mark the host blocked, tell the owner once, probe every 15 minutes, keep accepting stimuli.
- **Refusals:** recorded, then the next turn.
- Waits are recorded as degraded time, not chosen rest.

Safety valves, not budgets

VALVE	RULE	WHEN IT TRIPS
Spending cap	A per-day dollar cap, for paid providers only. Off for free and local ones.	Halts the host; the owner restarts it
Turn-rate ceiling	A generous maximum number of turns per minute, as a guard against a runaway bug	Waits briefly, recorded as degraded
Grounding streak	Several self-directed turns in a row with no tool use, worker, change to work items or message	Forces an integrity turn
Repetition guard	Turn output or the carried note nearly copies recent ones	Forces an integrity turn and tells the owner

Prove the loop that never rests, at no cost

Slice 0 splits the agent library with no change in behaviour. Slice 1 builds the host against a scripted mock model, a fake world and injected provider faults. The gates are fixed before the first run.

Eleven gates

GATE	PASSES WHEN
No rest	Over 30 quiet minutes, under 2% of the time is spent outside a turn, apart from declared degraded time. The next turn starts within 50 ms of the last one ending, p99.
Responsive	Owner messages are admitted within one turn's duration, p95. No top turn runs past its bound.
Fair pulls	Every live pull is served within 12 self-directed turns. No pull takes more than half of any 50-turn window.
Schedule	Due items enter at the next boundary. Items missed while down fire once, marked late.
Expectations	Only the host can settle one. Calibration recomputed from the record matches exactly.
Workers	Three or more run while the top keeps turning. Each carries a verdict from the host. Workers cannot commission.
Provider faults	The process never exits. Requeued stimuli are admitted exactly once, and requested waits are honoured.
Stop	A halt within the remaining mock call plus 1 s, or within 1 s from a wait. The watchdog catches a wedged loop.
Hard kill	After 50 random kills, every stimulus has exactly one outcome, the rebuilt state matches, and the gap is stated.
Bounded	Over 10,000 turns, context stays flat once full and never exceeds its budget.
Cost	Zero: no network beyond loopback, no live model.

Main risks

- **Rumination:** a loop that cannot rest copies itself. Grounding duty, repetition guard and integrity turns stand against it, but they are untested on a real model.
- **Gaming the arbiter:** the agent shapes its own work items to win the turns it likes.
- **Expectation theatre:** only safe bets get made.
- **Free-provider starvation:** rate limits keep the host degraded much of the time.
- **Runaway delegation:** capped by worker count, ticket budgets and depth one.
- **Exposure** of an always-on endpoint: loopback by default, role tokens, owner-only stop.
- **Disk growth** of a record that is never deleted, only rotated.

Size, inferred: slice 0 is about 1,000 lines changed, slice 1 about 3,000 to 4,000 with tests. The protocol and the real engine come in slice 2; a live run in slice 3.

Three questions, most blocking first

Each is one decision, with my recommendation and what the answer changes. Everything else I decided myself; those choices are listed alongside so any can be overruled.

1. Build slices 0 and 1 now?

May the agent library be split into a clean library, with no change in behaviour, and the host then built against a mock model, at no cost and behind the gates?

Recommendation. Yes. Both are free, reversible and gated, and nothing reaches a live model.

What the answer changes. Whether the held first-slice task is released to build, or the design waits.

2. First live run: a local GPU or a free hosted router?

Which substrate hosts the first real run?

Recommendation. A local GPU behind a standard chat endpoint, with the free router as fallback. A loop that never rests meets free-tier rate limits constantly. A local model also exposes token probabilities for real surprise.

What the answer changes. Which machine runs slice 3, the backoff defaults, whether token-level surprise enters the register, and whether the host shares a GPU with other work.

3. May the agent change the world itself, or only through workers?

Should the always-on agent hold write, shell and web tools itself?

Recommendation. No. Give it reading, memory, its work items, messages and commissioning only. Every change to the world goes through a worker whose result the host checks.

What the answer changes. The top agent's default tools, the safety envelope of an agent no one watches, and whether the host check covers every effect on the world or only delegated work.

Decided by me

- Name and place: the host crate lives in the rung workspace.
- Stimuli enter only at the next turn boundary.
- One agent, many protocol channels. Turns may be shared, each prompt still gets exactly one answer, and agent-initiated messages go to an opt-in outbox.
- The host owns memory: baseline by default, an external provider by configuration.
- The calendar is the host's, and there is no wake tool.
- The arbiter is scored, logged and swappable. The agent may defer, never raise.
- The agent never settles its own expectations.
- Workers: non-blocking, one level deep, checked by the host, never re-run on a crash.
- Stop and watchdog work outside the loop. There is no pause state.
- Spending caps only for paid providers.
- The seed identity and first projects are the operator's configuration, not rung code.